



University of Pisa

Department of Information Engineering

Master's Degree Artificial Intelligence and Data Engineering

Internet of Things: Smart Health Monitoring

Professors:

Giuseppe Anastasi

Francesca Righetti

Carlo Vallati

Students:

Mohsin Ali

Andrea Zanin

ACADEMIC YEAR 2025/2026

Contents

1	Introduction & System Architecture	1
1.1	System Overview & Use Case	1
1.2	System Architecture	1
1.3	Protocol Justification (MQTT vs. CoAP)	2
2	TinyML Edge Diagnostics & Risk Profiling	3
2.1	Model Training & Hardware Optimization	3
2.2	Heterogeneous Patient Profiling	3
2.3	Model Evaluation	3
3	System Implementation & Closed-Loop Logic	5
3.1	Sensor Node Firmware & Hardware Integration	5
3.2	IPv6 SLAAC Address Resolution	5
3.3	Database Schema	5
3.4	Logical Workflow & Closed-Loop Control	6
4	Evaluation & Stress Testing	7
4.1	Baseline Performance & Channel Jamming Stress Test	7
4.2	Adaptive Mitigation & Experimental Results	7
5	Conclusions and Future Work	8

Introduction & System Architecture

1.1 System Overview & Use Case

This project introduces a clinical IoT monitoring framework for a **Smart Health Hospital Ward**, providing real-time physiological supervision and automated alerting. Patients wear wireless nRF52840 sensor nodes that monitor heart rate (BPM). Instead of relying on static thresholds, the system employs **edge intelligence (TinyML)**:

- **Local ML Prediction:** A Random Forest classifier on each node predicts patient-specific cardiovascular disease risk (0=Normal, 1=Anomaly) by fusing real-time BPM with static clinical features (Age, Gender, Blood Pressure, Cholesterol, Diabetes).
- **Actuation & Nursing Protocol:** Upon anomaly detection, the node's LED turns Red and alerts the backend, which immediately issues a CoAP command to latch the alarm. For medical safety, the alarm requires physical intervention: nurses must assess the patient bedside and manually clear the state via a **3-second button press** on the sensor, resetting the latch and LED.

1.2 System Architecture

The architecture coordinates five distinct layers:

1. **Edge Layer:** nRF52840 nodes running Contiki-NG for HR collection and local TinyML inference.
2. **Wireless Mesh Layer:** IEEE 802.15.4/MAC with RPL-Lite routing (operating on Channel 26, PANID 0xabcd).
3. **Gateway Layer:** An nRF52840 Border Router bridging wireless to IP via a SLIP tunnel (`tunslip6`) onto the host's virtual interface (`tun0`).
4. **Middleware & DB Layer:** A Mosquitto MQTT broker routes JSON telemetry to a Python backend, which evaluates and stores structured records in a MySQL database (`smart_health_db`).
5. **Presentation Layer:** A Grafana dashboard visualizing real-time health telemetry and network metrics (PDR, CoAP latency).

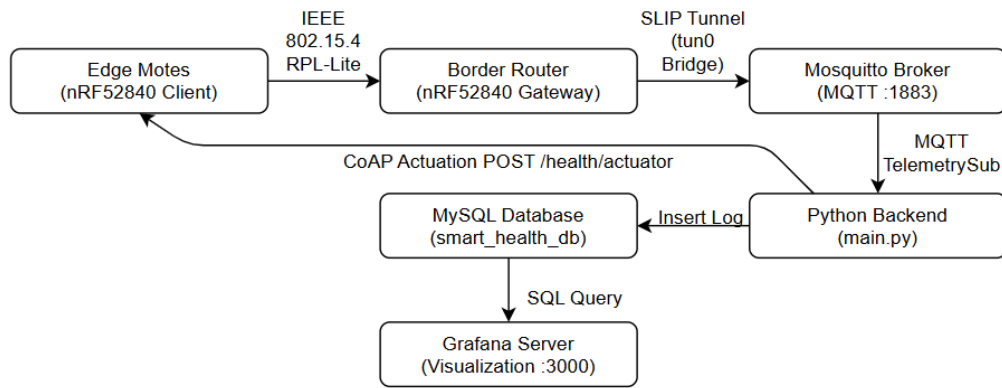


Figure 1.1: Smart Health IoT System Architectural Schema

1.3 Protocol Justification (MQTT vs. CoAP)

Integrating both **MQTT** and **CoAP** [1] is a core design requirement:

- **MQTT (TCP, Publish-Subscribe)** ensures reliable, continuous upstream streaming of periodic health telemetry from the nodes to the backend.
- **CoAP (UDP, Request-Response)** manages downstream actuations and on-demand configurations. Eliminating the overhead of persistent TCP sockets makes it highly efficient for low-latency commands on battery-constrained nodes.

TinyML Edge Diagnostics & Risk Profiling

2.1 Model Training & Hardware Optimization

The edge diagnostic system employs a Random Forest classifier trained on the Kaggle **Heart Disease Dataset** [2] using `scikit-learn` (75% accuracy). To comply with the nRF52840 memory constraints, inference relies on six features: `Age`, `Sex`, `RestingBP`, `Cholesterol`, `FastingBS`, and `MaxHR` (dynamically mapped to the live BPM reading).

The model was exported to standard C code via `m2cgen`. However, the raw output relies on double-precision (`double`) math. Executing this on the nRF52840's ARM Cortex-M4F triggers slow software emulation, risking CPU starvation and memory alignment faults within Contiki-NG [3]. By manually casting all variables and arrays in `heart_disease_model.h` to single-precision `float`, the system leverages the hardware Floating-Point Unit (FPU), reducing inference time to micro-seconds and ensuring RTOS stability.

2.2 Heterogeneous Patient Profiling

To simulate a realistic ward, nodes dynamically load specific clinical profiles based on their 16-bit MAC address suffix at boot. This validates the edge intelligence paradigm: identical live heart rates yield completely patient-specific anomaly decisions.

- **d8c7 (Node 2 – High-risk Male, 72y)**: RestingBP 150, Chol 270, FastingBS 1. Clinically fragile.
- **92a7 (Node 3 – Healthy Female, 28y)**: RestingBP 110, Chol 175, FastingBS 0. Clinically resilient; anomaly status remains normal even at elevated HR.
- **4d7c (Node 4 – Mod-risk Male, 48y)**: RestingBP 135, Chol 220, FastingBS 0. Moderate profile.
- **f869 (Node 5 – Mod-risk Female, 65y)**: RestingBP 140, Chol 240, FastingBS 1. Moderate risk.

2.3 Model Evaluation

The classifier's performance was assessed on a held-out validation split.

Table 2.1: Classification report for the validation set (0 = No Disease, 1 = Disease).

Class	Precision	Recall	F1-Score
0 (No Disease)	0.68	0.73	0.70
1 (Disease)	0.81	0.76	0.79

The metrics reflect a solid recall (0.76) for Class 1. In a clinical alerting context, this relatively low false-negative rate is a strict requirement, as missed physiological anomalies carry a vastly higher operational risk than occasional false alarms.

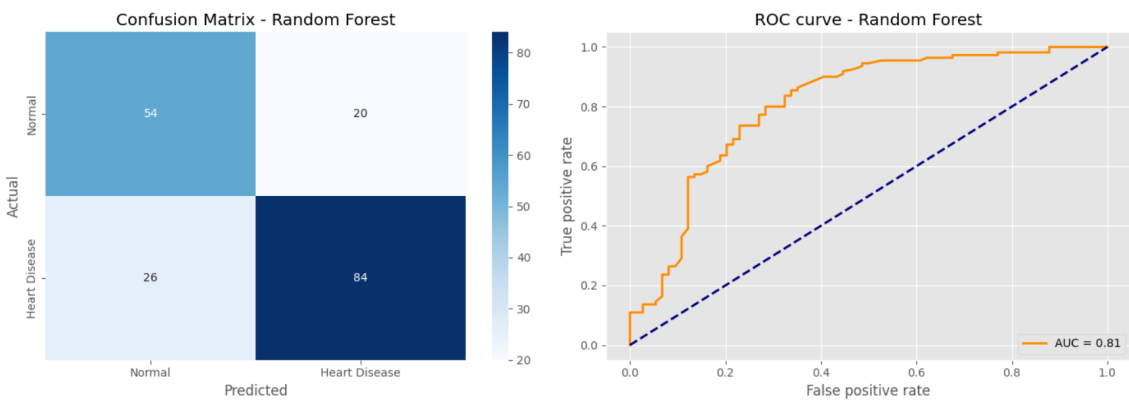


Figure 2.1: Confusion matrix of the Random Forest classifier on the validation set.

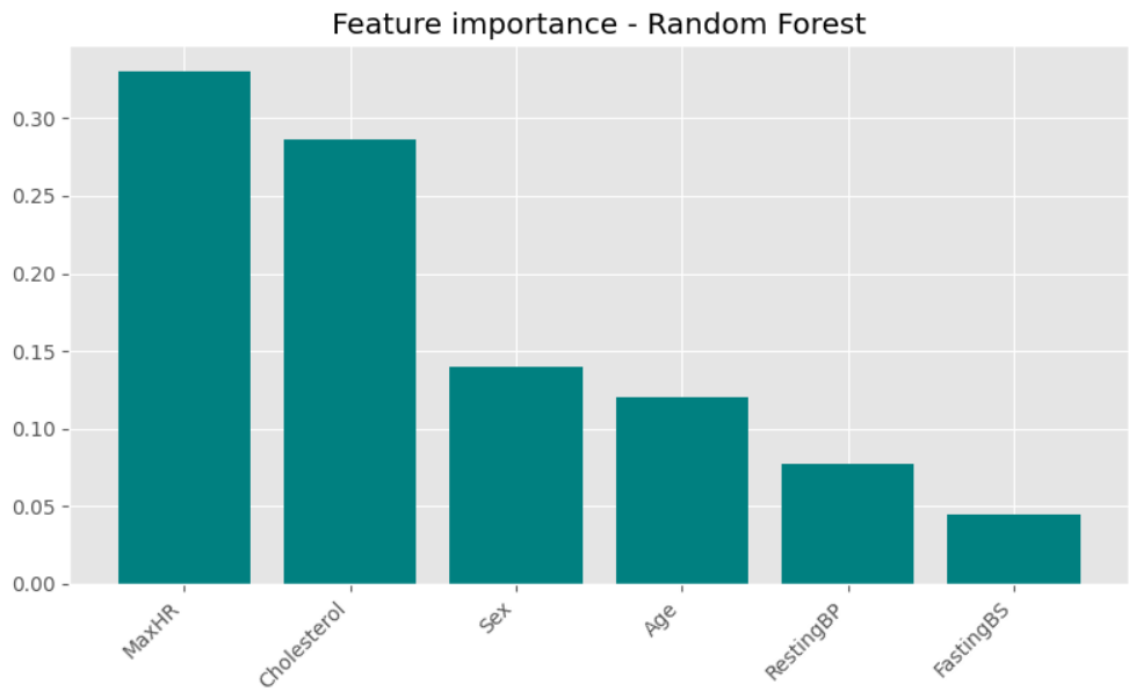


Figure 2.2: Features importance graph.

System Implementation & Closed-Loop Logic

3.1 Sensor Node Firmware & Hardware Integration

Built on Contiki-NG's event-driven protothread model, the client node firmware manages:

- **MQTT Client:** Registers via a JSON packet to `health/node/register` and subscribes to `health/node/actuator` for upstream telemetry.
- **CoAP Server:** Exposes `health/status` (GET, read-only vitals and anomaly states) and `health/actuator` (POST/PUT). The latter accepts `mode=on/off` for manual LED latching and `mode=congestion_on/off` to toggle telemetry suppression.
- **Hardware I/O:** Integrates the dongle's `button-hal` and `leds-hal`. Button states map directly to emergency workflows (detailed in Section 3.4).

3.2 IPv6 SLAAC Address Resolution

Devices autoconfigure IPv6 addresses dynamically via SLAAC (EUI-64). Simulated Cooja motes construct padded Interface Identifiers (IIDs) (e.g., `fd00::202:2:2:2`), while physical nRF52840 nodes map their factory MAC address suffix (e.g., `d8:c7` becomes `fd00::2000:d8c7:d8c7:d8c7`).

To ensure theoretical alignment and seamless cross-testbed communication, the Python backend employs a dual address-resolution pipeline:

1. It dynamically scrapes the Border Router's HTTP routing table to resolve active SLAAC addresses.
2. If the router interface is unreachable, it falls back to a deterministic mapping that mathematically translates the node's MAC suffix into its standard IID segment.

3.3 Database Schema

Historical logging is centralized in a MySQL database (`smart_health_db`) using four core schemas:

- **nodes_directory:** Tracks registered devices, their domain, and protocol.
- **telemetry_history:** Time-stamped logs of every status update (`seq`, `HR`, `anomaly flag`, `user alert flag`).
- **closed_loop_control_log:** Records backend control decisions, targeting `node_id`, `anomaly triggers`, and `execution timestamps`.
- **network_metrics:** Stress-test evaluations, storing `Packet Delivery Ratio (pdr)`, `CoAP round-trip latency`, and `active congestion states`.

3.4 Logical Workflow & Closed-Loop Control

To provide a complete operational overview, the system lifecycle executes through three structured stages:

1. **Deployment & Boot:** The FPU-optimized firmware is flashed via the nRF52840 USB bootloader. On power-up, nodes load specific patient baseline profiles based on their MAC suffix. They connect to the MQTT broker, publish registration data, and the Python backend updates the SQL directory after resolving the IPv6 address.
2. **Continuous Edge Intelligence:** Every 5 seconds, nodes generate HR readings, execute the TinyML Random Forest classifier, and stream telemetry. If a patient experiences physical distress not captured by HR, a **short button press** toggles `user_alert_active=1`, flashing the Red LED and publishing a QoS 1 emergency alert bypassing rate-limits.
3. **Closed-Loop Automated Actuation:** The Python backend continuously parses incoming MQTT telemetry. If a node flags an ML anomaly (`anomaly = 1`) or a manual alert, the backend logs the event and asynchronously dispatches a CoAP POST (`mode=on`) directly to the node. The node responds by disabling Green indicators and latching its physical Red LED to ON. This critical medical alert persists indefinitely until a nurse physically assesses the patient and executes a **3-second long button press** to override and clear the state.

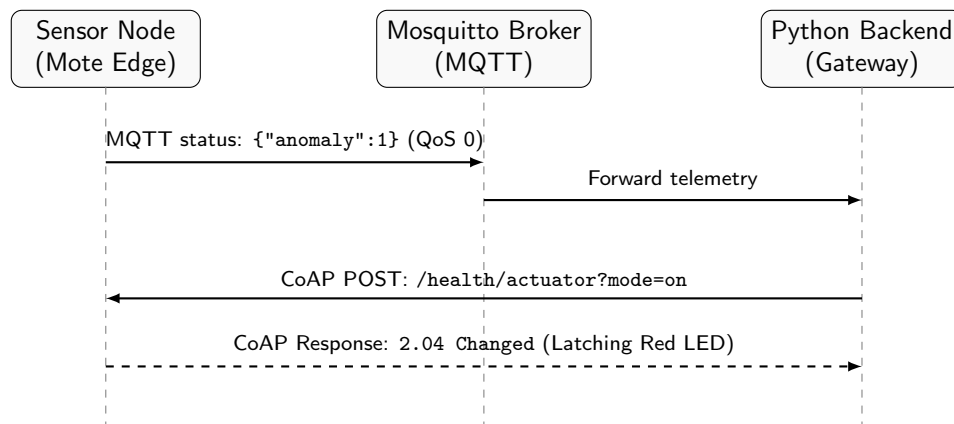


Figure 3.1: Closed-Loop control feedback sequence diagram.

Evaluation & Stress Testing

4.1 Baseline Performance & Channel Jamming Stress Test

Under nominal conditions, the 802.15.4 mesh operates stably: patient telemetry streams every 5 seconds, CoAP round-trip time (RTT) remains low (30–120 ms), and the MQTT Packet Delivery Ratio (PDR) is 100.0%.

To evaluate network resilience, a **Spammer Node** was deployed on the same PANID (0xabcd, Channel 26), broadcasting a burst of 5 multicast UDP packets every 10 ms (500 pkts/sec). This flood aggressively saturates the radio frequency, forcing CSMA/CA MAC layer collisions and transmission back-offs. Consequently, the CoAP diagnostic RTT immediately spikes to **400–600 ms** (with timeout peaks up to 2000 ms), successfully simulating a severely congested ward.

4.2 Adaptive Mitigation & Experimental Results

To counter radio saturation, the system implements two concurrent closed-loop mechanisms:

- **Edge Adaptation (Rate Backoff):** Upon local TinyML anomaly classification, the node throttles its reporting interval from 5s to 20s. While clinically counter-intuitive (as medical emergencies typically demand higher resolution), this strict network-centric mechanism drastically reduces the alarming node’s channel footprint, reserving bandwidth for other critical mesh traffic.
- **Gateway Adaptation (Deadband Filter):** A Python background monitor continuously evaluates CoAP RTT. If the average latency exceeds **300 ms**, the backend dispatches a CoAP `mode=congestion_on` command to all nodes. Nodes immediately suppress stable telemetry, publishing only if the heart rate shifts by ≥ 4 BPM or if the anomaly status changes. This filters up to 80% of redundant traffic. Once latency recovers (≤ 200 ms), the gateway restores periodic 5-second reporting via `mode=congestion_off`.

Experimental Outcomes: The closed-loop trials confirmed that without adaptation, continuous periodic transmissions during jamming caused perpetual congestion. The adaptive deadband filter successfully decreased channel occupancy, allowing CoAP latency to dynamically recover below the 200 ms threshold.

Crucially, due to the inherent reliability of TCP (which handles dropped packets via automatic retransmissions), the MQTT PDR remained artificially stable at 100.0% throughout the jammer attack. Thus, the system proves that **CoAP UDP latency is a vastly superior and more reactive metric** for detecting physical RF congestion than TCP PDR in clinical IoT networks.

Conclusions and Future Work

This project demonstrates a complete IoT pipeline for smart health monitoring, from constrained-device machine learning inference to cloud-side closed-loop network adaptation. The integration of edge TinyML diagnostics allowed anomaly detection to remain functional even under degraded network conditions, while the dual-protocol architecture (MQTT/CoAP) balanced reliability and low-latency control. The stress-testing phase confirmed that CoAP round-trip latency is a substantially more sensitive congestion indicator than MQTT-based PDR in TCP-backed transports, and that the combination of local rate backoff and gateway-triggered value-based reporting is sufficient to recover the network from a congested to a stable state without manual intervention.

Future Work

Possible extensions include:

- **Scalability Evaluation:** replicating the stress tests across a larger number of concurrent patient nodes to evaluate scalability.
- **Over-the-Air Retraining:** introducing a persistence layer for TinyML model updates.
- **Dynamic Actuation Logic:** extending the closed-loop actuation logic to modulate CoAP polling frequency dynamically rather than using fixed latency/PDR thresholds.

Bibliography

- [1] A. Bandilet *et al.*, “Performance comparison of mqtt and coap protocols in iot applications,” *IEEE Internet of Things Journal*, vol. 8, no. 4, pp. 3012–3021, 2021.
- [2] fedesoriano, “Heart failure prediction dataset.” Hosted on Kaggle. <https://www.kaggle.com/datasets/fedesoriano/heart-failure-prediction>, 2021. Original data from UCI Machine Learning Repository (Janosi, Steinbrunn, Pfisterer, Detrano).
- [3] A. Dunkels *et al.*, “Contiki-ng: The open source operating system for next-generation iot devices,” *ACM Transactions on Internet of Things*, vol. 1, no. 2, 2020.